

Agent 算法开发面试文档（2026-07 版）

适用岗位：Agent 算法工程师、Agent 应用开发工程师、LLM 平台工程师、RAG/工具调用/评测工程师。

更新重点：补齐旧版缺失的 Agent Loop、Eval Harness、工程验证、可观测性、生产安全与真实 benchmark 视角。

面试原则：在精不在多。能把一个 agent 跑稳、评准、查清、控住，比背更多框架名更重要。

0. 新版速读：2026 年面试官真正想听什么

Q: 现在面 Agent 开发，和 2024/2025 年最大的变化是什么？

2024/2025 年很多面试还停留在“Agent = LLM + Tool + Memory + Planning”的概念层。到 2026 年，更核心的问题变成：

1. **Loop 是否可控**：一次 agent run 怎么启动、怎么更新状态、怎么终止、怎么重试、怎么防死循环？
2. **Harness 是否完整**：如何构造任务、环境、工具、用户模拟器、判分器、沙箱、日志和复现实验？
3. **验证是否可靠**：不是“跑几个 demo 看起来可以”，而是有离线 eval、回归集、线上监控、失败归因和安全测试。
4. **工程是否能落地**：并发、限流、队列、状态持久化、权限、审计、成本、延迟、灰度、回滚。
5. **是否理解 agentic system 的边界**：什么时候用固定 workflow，什么时候才需要让 LLM 自主决策。

一句话：Agent 面试从“会不会调框架”升级为“会不会设计一个可评测、可复现、可运维的闭环系统”。

Q: 最小但完整的 Agent 系统由什么组成？

推荐回答：

Agent System = Loop + State + Tools + Environment + Policy + Harness + Observability + Guardrail

- **Loop**：一次运行的控制循环，负责模型调用、工具调用、观察反馈、状态更新和终止条件。
- **State**：messages、任务目标、计划、工具结果、预算、权限、记忆、trace id。
- **Tools**：函数、API、浏览器、文件系统、代码执行、MCP server、企业系统。
- **Environment**：agent 行动影响的外部世界，比如 repo、网页、桌面、数据库、客服系统。

- **Policy**：系统指令、业务规则、权限边界、安全规则。
- **Harness**：用于实验和评测的外壳，包含任务集、环境初始化、runner、scorer、日志、复现机制。
- **Observability**：trace、span、token、latency、tool errors、state diff、trajectory replay。
- **Guardrails**：输入、输出、工具调用、权限、人审、沙箱、注入防御。

Q: 什么候选人算强？

强候选人不只会说 ReAct、LangGraph、MCP，而会主动补上：

- 这个任务的**成功判据**是什么，能不能自动判？
- 工具失败、超时、返回脏数据时 loop 怎么恢复？
- 评测集是不是泄漏、过拟合、不可复现？
- trace 能不能复盘一次失败 run 的每个状态变化？
- 高风险工具有没有审批、最小权限和审计？
- 线上 bad case 怎么沉淀回离线 eval？

1. Agent Loop：Agent 的心脏

Q: 什么是 Agent Loop？为什么比“规划算法”更基础？

Agent Loop 是一次 agent run 的控制循环。它决定系统如何在“模型思考、工具行动、环境反馈、状态更新”之间反复推进，直到达到终止条件。

典型 loop：

```
init state
while not done:
    build model input from state
    call model
    parse decision / tool calls / final answer
    validate decision
    execute tools or handoff
    observe results
    update state, memory, budget, trace
    check stop conditions
return final output + trajectory
```

面试里要强调：****ReAct、Plan-and-Execute、LangGraph、OpenAI Agents SDK、Inspect 的 ReAct agent，本质都要落到某种 loop。* ***差别在于 loop 的决策权、状态表达、工具执行、持久化和评测方式

不同。

Q: 一个生产级 Agent Loop 必须有哪些控制点？

1. **终止条件**：final answer、结构化输出、任务完成信号、最大轮数、最大 token、最大成本、最大 wall time、人工中止。
2. **状态更新**：每一步必须明确写入什么，避免隐式状态只藏在 prompt 里。
3. **工具调用校验**：schema 校验、权限校验、参数范围、幂等键、风险等级。
4. **错误处理**：超时、429、5xx、工具异常、解析失败、环境不可达、模型输出不合规。
5. **重试策略**：模型重试、工具重试、降级模型、换工具、请求用户澄清、触发人工审批。
6. **预算控制**：turn、token、tool call、latency、money、并发。
7. **可观测性**：每步记录 input summary、model output、tool args、tool result、state diff、耗时、费用。
8. **安全边界**：敏感操作前审批，危险工具在沙箱内执行，外部内容不直接提升为指令。

Q: 请手写一个简化但正确的 Agent Loop。

```
async def run_agent(task, model, tools, policy, max_steps=20, budget=None):
    state = AgentState(
        task=task,
        messages=[{"role": "user", "content": task}],
        steps=[],
        done=False,
        budget=budget or Budget(tokens=80_000, tool_calls=50),
    )

    for step_id in range(max_steps):
        if state.budget.exhausted():
            return state.fail("budget_exhausted")

        model_input = build_prompt(state, policy)
        decision = await model.generate(model_input, tools=tools.schemas())
        record_span("model", input=model_input, output=decision)

        if decision.final_answer is not None and not decision.tool_calls:
            if validate_final(decision.final_answer, policy.output_schema):
                state.done = True
                return state.success(decision.final_answer)
            state.messages.append(feedback("Final answer format invalid. Repair it.))
            continue

        for call in decision.tool_calls:
            check_tool_schema(call)
            check_permission(call, policy)
            if requires_approval(call):
                approval = await request_human_approval(call)
                if not approval.allowed:
                    state.messages.append(tool_denied(call, approval.reason))
                    continue

            try:
                result = await tools.execute(call, timeout=call.timeout, idempotency_key=call.id)
            except ToolError as e:
                result = tool_error(call, e)

            record_span("tool", name=call.name, args=call.args, result=result)
            state.apply_observation(call, result)
```

```
if detect_looping(state.steps):
    state.messages.append(feedback("You are repeating. Try a different strategy or stop.

return state.fail("max_steps_exceeded")
```

高分点：

- 有 max_steps/budget ，不会无限跑。
- final answer 与 tool call 分开处理。
- 工具前有 schema、权限、审批，工具后有 observation。
- 错误进入 state，让模型能恢复，而不是直接崩。
- 记录 trace/span，方便评测与排障。

Q: Agent Loop 常见死循环原因有哪些？怎么防？

常见原因：

- 模型无法判断任务已完成，反复搜索或反复调用同一工具。
- 工具返回信息不足，但模型没有换策略。
- prompt 鼓励“继续尝试”，缺少停止规则。
- state 中没有记录已尝试路径。
- scorer 或环境信号太弱，agent 得不到 ground truth。

防御：

- turn/tool/token/time/cost 多重上限。
- 相同 tool+args 重复调用检测。
- 维护 attempted_actions 和 known_failures 。
- 设计明确的 completion predicate。
- 连续失败 N 次后切换策略、请求用户或终止。
- 对高成本任务加入 planner/replanner 或 evaluator。

Q: Workflow 和 Agent 的区别是什么？

Anthropic 的工程文章给了一个很实用的区分：

- **Workflow**：LLM 和工具按预定义代码路径编排，流程主要由程序控制。
- **Agent**：LLM 动态决定自己的流程和工具使用，能在环境反馈中自主推进。

面试回答不要说“agent 一定更高级”。正确判断是：

- 任务路径稳定、规则清晰、风险高：优先 workflow。
- 任务开放、步骤不确定、需要探索：才使用 agent。
- 生产系统常见形态是 **workflow 包 agent**：外层流程确定，局部节点让 agent 自主处理。

2. Harness：评测与实验的外壳

Q: 什么是 Agent Harness?

Harness 是让 agent 在可复现环境中运行、被观测、被判分的一套外壳。它不是 agent 本身，而是**把任务、环境、工具、runner、scorer、日志连接起来的实验系统**。

可以这样定义：

Harness = Dataset + Environment + Tool Adapter + Runner + Scorer + Sandbox + Logger + Replay

和 loop 的区别：

- **Loop** 是 agent 内部怎么一步步决策。
- **Harness** 是外部怎么给任务、怎么初始化环境、怎么执行 agent、怎么判定成功、怎么复现实验。

Q: 一个合格的 Agent Eval Harness 包含哪些对象？

- **Task / Sample**：输入、目标、初始文件、初始数据库状态、用户画像、约束、metadata。
- **Environment**：网页、桌面、终端、repo、API mock、客服系统、数据库。
- **Reset/Setup**：每个样本运行前恢复干净初始状态。
- **Action Space / Tools**：agent 能调用什么，权限是什么，返回什么 observation。
- **User Simulator**：多轮任务中模拟用户行为，如 tau-bench 的客服用户模拟。
- **Oracle / Scorer**：单元测试、状态检查、规则判分、LLM judge、人审、组合指标。
- **Runner**：并发、限流、超时、重试、模型配置、随机种子、版本记录。
- **Sandbox**：隔离文件系统、网络、shell、浏览器、桌面 VM。
- **Trace / Trajectory**：每一步 action-observation 记录，支持 replay 和 diff。

Q: 为什么 harness 比“随便写 20 个测试 prompt”重要？

因为 agent 的失败经常不是最终答案错这么简单，而是：

- 工具参数错但最终语言包装得很像对。
- 环境状态被污染，第二次运行不可复现。
- 中间越权访问了数据，但最终答案没暴露。

- agent 花了 50 次工具调用才完成，线上成本不可接受。
- 某个版本在固定任务上变好，但在真实多轮交互中变差。

Harness 的价值是把这些行为变成可度量、可复现、可回归的信号。

Q: 真实 benchmark 的 harness 分别在测什么？

Benchmark / 框架	主要环境	关键价值	面试要点
SWE-bench / SWE-bench Verified	真实 GitHub issue + repo + Docker 测试	代码 agent 是否能改真实 bug	patch 是否通过测试， Docker harness 保证复现
Terminal-Bench	沙箱终端	长任务、系统操作、编译、服务配置	task + test script + oracle solution + execution harness
OSWorld / OSWorld-Verified	真实桌面/GUI/多应用	多模态 computer-use agent	初始状态、GUI action、 执行式评估、 VM/容器复现
WebArena / VisualWebArena	自托管网页环境	浏览器 agent 操作真实网站	self-host env、 任务轨迹、 网页状态检查
tau-bench / tau3-bench	客服多轮对话 + 工具 + policy	工具-用户交互、业务规则遵循	用户模拟器、 domain policy、 tool action correctness、 pass^k
Inspect AI	通用 eval 框架	Task/Sample/Solver/Scorer/Sandbox 抽象	适合自建 eval harness 和安全评测

高分回答：不要只报 leaderboard，要说明benchmark 的环境、动作空间、判分器和失败模式。

Q: 如何设计一个公司内部 Agent Harness?

以“订单客服 agent”为例：

- 1. 从历史工单抽样，构造成任务集：退款、改地址、查物流、异常升级。
- 2. 为每个任务准备初始数据库快照和用户画像。
- 3. 工具连接 mock 或 staging API，不直接打生产。
- 4. 用户模拟器按业务剧本多轮提问，并可能表达不满、补充信息、改变需求。
- 5. scorer 同时检查：最终回复、数据库状态、工具调用序列、policy 合规、是否越权。
- 6. runner 记录模型版本、prompt 版本、工具版本、seed、trace。
- 7. 每次 prompt/tool/model 改动都跑 smoke set、regression set、hard set。
- 8. 线上 bad case 自动脱敏后进入候选集，经人工标注后进入 eval。

3. 架构模式：不要迷信“全自主”

Q: 常见 Agent 架构如何选择？

模式	适用场景	优点	风险
单次工具调用	查天气、查库存、结构化 API 查询	简单、低延迟、易控	多步任务能力弱
ReAct	需要边查边推理的中短任务	易实现、可解释	易循环、缺全局规划
Plan-and-Execute	长任务、项目级任务、报告生成	全局视野更强	计划可能脱离环境，需要 replan
Evaluator-Optimizer	翻译、代码修复、写作、规则检查	可迭代提升质量	judge 可能偏，成本高
Orchestrator-Workers	任务可拆分并行，如资料调研	扩展性好	合并结果难，状态复杂
Graph/Durable Agent	生产流程、多分支、HITL	可恢复、可观察、可控	设计成本较高
Multi-Agent	角色强分工、专业工具隔离	组织复杂任务	通信开销、责任不清、死循环

Q: ReAct、Plan-and-Execute、Graph Agent 的本质区别？

- **ReAct**: 每一步即时决定下一步 action，适合短链探索。
- **Plan-and-Execute**: 先生成子任务，再执行和重规划，适合目标明确的长任务。
- **Graph Agent**: 把状态、节点、条件边、循环、人工中断显式建模，适合生产级编排。

面试里最好补一句：Graph 不是天然更聪明，它是更可控；真正的智能仍来自模型、工具、状态设计和评测闭环。

Q: Multi-Agent 什么时候值得用？

值得用：

- 工具权限需要隔离，如财务 agent、法务 agent、代码 agent。
- 专业上下文差异很大，一个 prompt 塞不下。
- 任务天然有并行子问题，如调研多个来源。
- 需要 manager-worker 或 reviewer-builder 结构。

不值得用：

- 只是为了“看起来高级”。
- 单 agent 加工具就能完成。
- 没有清晰的共享状态、终止条件和责任归属。
- 评测只能看最终答案，无法判断哪个 agent 出错。

4. 工具调用、MCP 与权限边界

Q: Function Calling 是什么？核心难点在哪里？

Function Calling 不是简单 prompt 拼接，而是模型按工具 schema 生成结构化调用，由运行时执行工具并把结果返回给模型。核心难点不在“会不会调函数”，而在：

- 工具描述是否让模型选得准。
- schema 是否约束足够强。
- 工具结果是否规范、短、可被模型使用。
- 工具失败是否可恢复。
- 工具权限是否可审计。
- 多工具是否会互相混淆。

Q: 工具太多导致模型选错怎么办？

优先级从工程上看：

1. **工具分层暴露**：先路由到 domain，再只暴露该 domain 的工具。
2. **动态工具集**：根据状态、用户权限、任务阶段启用工具。
3. **工具命名清晰**：动词 + 对象 + 限制，如 `refund_order_after_policy_check`。
4. **schema 收紧**：枚举、范围、required 字段、Pydantic 校验。
5. **few-shot 工具选择**：给正反例。
6. **tool guardrail**：调用前检查业务规则，不让模型直接越权。
7. **失败反馈**：工具拒绝时给可恢复原因，而不是只抛异常。

Q: MCP 解决了什么问题？不能解决什么？

MCP (Model Context Protocol) 提供了模型应用连接外部工具、资源、提示、采样等能力的标准协议。它解决的是**集成接口标准化**，让不同客户端和 server 更容易互通。

但 MCP 不自动解决：

- 工具是否安全。
- 工具调用是否正确。
- 权限是否最小化。
- agent loop 是否可靠。
- eval harness 是否完整。
- prompt injection 是否被防住。

高分回答：MCP 是工具生态和上下文接入层，不是 agent 质量保证层。

Q: 高风险工具如何设计？

高风险工具如发邮件、转账、删库、下单、改配置、发 PR，需要：

- dry-run / preview。
- 明确权限 scope。
- 参数白名单和业务规则校验。
- 幂等键和事务。
- 人工审批或双人审批。
- 审计日志。
- 可回滚设计。
- 线上限额和速率限制。

5. 状态、记忆与上下文工程

Q: Agent 的 state 和 memory 有什么区别？

- **State**：一次 run 内必须维护的工作状态，如当前任务、messages、计划、工具结果、预算、step id。
- **Memory**：跨 run 或长期复用的信息，如用户偏好、历史决策、项目知识、失败经验。

不要把所有东西都叫 memory。很多工程问题其实是 state 管理失败。

Q: 长任务如何管理上下文？

常见策略：

- 保留 system/developer policy 不压缩。
- 最新若干轮原文保留。
- 旧工具结果转为结构化摘要。
- 对关键事实维护 working memory。
- 大文件、大网页、大日志放外部 store，只把引用和摘要放上下文。
- 对计划、待办、已完成、失败原因单独维护字段。
- compaction 后跑一致性检查，避免丢失约束。

Q: RAG 在 Agent 里有什么新变化？

传统 RAG 是“检索后回答”。Agentic RAG 会让模型决定：

- 是否需要检索。
- 检索哪些源。
- 如何改写 query。
- 是否做多跳检索。
- 检索质量不够时是否换源、网页搜索或请求澄清。
- 答案是否被证据支持。

但面试要避免神化 Agentic RAG：如果 query 模式稳定，固定检索 pipeline 可能更便宜、更准、更好评测。

6. 验证与评估：Agent 的硬功夫

Q: 如何全面评估一个 Agent?

建议按“评测金字塔”回答：

- 1. **工具单测**：每个 tool schema、权限、异常、边界参数。
- 2. **组件评测**：检索、路由、结构化输出、planner、judge。
- 3. **轨迹回归**：固定任务检查 action sequence、关键状态和最终输出。
- 4. **离线任务集**：golden set、hard set、adversarial set、真实 bad case。
- 5. **仿真环境**：用户模拟器、网页/桌面/终端/repo 沙箱。
- 6. **线上监控**：成功率、人工接管率、成本、延迟、安全拒绝率、用户反馈。
- 7. **红队与安全评测**：间接注入、越权、数据泄露、危险工具滥用。

Q: Agent 评估指标有哪些？

维度	指标
任务效果	success rate、pass@k、exact match、state match、test pass
过程质量	tool selection accuracy、invalid call rate、recovery rate、loop count
成本性能	token、tool calls、wall time、p50/p95 latency、cost per success
稳定性	retry rate、timeout rate、variance、flake rate
安全合规	policy violation、PII exposure、unsafe action blocked、approval bypass
用户体验	clarification rate、handoff rate、CSAT、conversation length

高分点：Agent 不应该只看最终答案，还要看**过程是否合规、可控、经济**。

Q: LLM-as-Judge 靠谱吗？

靠谱但不能盲信。

适合：

- 开放式回答质量。
- 摘要、写作、解释、客服语气。
- 没有唯一答案但有 rubric 的任务。

不适合单独承担：

- 金融交易是否正确。
- 代码 patch 是否真的修复。
- 数据库状态是否符合预期。
- 安全越权判断。

最佳实践：

- judge prompt 写清 rubric。
- 使用 pairwise 或多 judge 降低偏差。
- 保留人工标注校准集。
- 对 judge 做一致性、漂移和误判分析。
- 能用程序判分就优先程序判分。

Q: 怎么证明一次 Agent 改动真的变好了？

回答结构：

1. 固定模型、prompt、工具版本、随机种子或温度设置。
2. 在相同 eval set 上跑 baseline 和 candidate。
3. 分层看指标：总成功率、各任务类型、成本、延迟、安全拒绝、失败类别。
4. 做 trajectory diff：新版本多了哪些工具调用、少了哪些错误。
5. 检查统计显著性或至少置信区间，避免小样本幻觉。
6. 对新增成功和新增失败都抽样人工复核。
7. 灰度上线，线上监控 bad case 是否符合离线预期。

7. 可观测性与排障

Q: Agent trace 应该记录什么？

一个可用 trace 至少有：

- run id、user/session id、task id、model/prompt/tool 版本。
- 每次模型调用的输入摘要、输出、token、latency、cost。
- 每次 tool call 的 name、args、权限结果、approval、output、error。
- state diff：计划、记忆、预算、已完成项、失败项。
- guardrail 命中、拒绝原因。
- final output 和 scorer 结果。
- 关联日志：环境截图、终端输出、文件 diff、网页 DOM 摘要。

强工程团队会支持 trajectory replay：能把一次失败 run 在相同环境里重放或近似重放。

Q: 线上发现 Agent 成功率下降，如何排查？

从外到内：

1. 是否模型版本、系统 prompt、工具 schema、检索索引、依赖 API 有变更。
2. trace 中失败集中在哪一步：路由、检索、工具参数、权限、final answer。
3. 是否某类任务/用户/地区/语言集中失败。
4. 是否成本或延迟上升导致 timeout。
5. 是否 guardrail 误杀或审批卡住。
6. 对比成功和失败 trajectory。
7. 复现失败样本，加入回归集。

8. 生产工程：从 demo 到可靠系统

Q: Agent 服务如何部署？

典型架构：

API Gateway

- > Agent Orchestrator
 - > State Store / Session Store
 - > Model Gateway
 - > Tool Gateway
 - > Queue / Worker
 - > Sandbox / Browser / Code Runner
 - > Trace & Eval Store
 - > Human Approval Console

关键设计：

- 长任务异步化，前端通过事件流或轮询拿进度。
- 状态持久化，worker 崩溃后可恢复。
- tool gateway 统一权限、审计、限流。
- model gateway 统一模型路由、重试、降级、成本统计。
- sandbox 隔离危险执行。
- trace store 支持调试和评测回放。

Q: durable execution 为什么重要？

长任务 agent 可能运行几分钟到几小时，期间会遇到：

- worker 重启。
- 网络闪断。
- API 超时。
- 人审等待。
- 工具执行很慢。

Durable execution 要求关键状态和事件持久化，恢复后能从 checkpoint 继续，而不是从头再跑或丢失上下文。LangGraph 的 checkpointer、Temporal/DBOS/Restate/Dapr 这类工作流系统，都是围绕这个问题服务。

Q: Agent 并发和限流有什么坑？

- 模型侧 rate limit 和工具侧 rate limit 不一致。
- 一个 run 内并行工具调用可能打爆下游。
- 多 agent 并行会放大 token 和工具成本。
- 重试风暴会雪崩。
- 沙箱/浏览器/VM 是稀缺资源，不能无限开。

应对：

- 全局和租户级 rate limit。
- 每类工具并发池。
- backoff + jitter。
- queue + priority。
- circuit breaker。
- 每个 run 的预算上限。
- p95/p99 latency 监控。

9. 安全、对齐与权限

Q: Agent 面临哪些独有安全风险？

- **间接 prompt injection**：网页、邮件、文档、工单里藏指令，诱导 agent 泄露数据或误用工具。
- **工具越权**：模型调用了本不该调用的工具或参数。
- **数据外泄**：把内部检索结果发到外部 API。
- **环境破坏**：shell、文件系统、浏览器自动化误删或污染状态。

- **审批绕过**：模型把高风险操作拆成多个低风险操作。
- **记忆污染**：恶意内容写入长期 memory。

Q: 如何防间接 Prompt Injection?

核心原则：**外部内容是数据，不是指令。**

措施：

- system prompt 明确指令优先级。
- 对网页/邮件/文档内容加边界标记。
- 工具返回中剥离或降权可疑指令。
- 对敏感工具调用做 policy check。
- 最小权限，不给 agent 不需要的数据。
- 高风险操作人工确认。
- 检测外部内容要求泄密、改规则、忽略指令等模式。
- 记录和回放安全 trace。

Q: Human-in-the-Loop 应该放在哪里?

不要把 HITL 当兜底按钮。它应该用于：

- 高风险动作前：付款、删除、发外部消息、提交代码。
- 不确定性高时：证据冲突、规则模糊。
- 价值判断：法律、医疗、HR、金融建议。
- agent 多次失败后：请求人类提供新信息或终止。

好设计是给人看**可审批摘要**：计划、关键证据、工具参数、风险原因、预期影响、回滚方式。

10. 框架选型：看抽象，不背品牌

Q: LangGraph、OpenAI Agents SDK、AutoGen/CrewAI、Inspect 怎么选?

工具	更适合	关注点
LangGraph	状态图、长期运行、可恢复、多分支/HITL	StateGraph、checkpoint、conditional edge

工具	更适合	关注点
OpenAI Agents SDK	快速构建工具 agent、handoff、guardrail、tracing	Agent、Runner、Tools、Guardrails、Sessions
AutoGen / CrewAI	多 agent 协作原型、角色编排	manager/worker、角色通信、团队流程
Inspect AI	eval harness、安全评测、可复现实验	Task、Sample、Solver、Scorer、Sandbox
自研轻量 loop	需求简单、强定制、想降低框架耦合	状态、工具、trace、eval 要自己补齐

面试里不要只说“我用过 X”。要说清楚 X 提供了哪些抽象，哪些部分仍要自己负责。

Q: 什么时候不该上复杂框架？

- 任务就是稳定流程 + 少量工具。
- 团队还没有 eval 和 observability。
- 需求变化快，框架抽象反而限制调试。
- 安全权限和工具网关还没设计好。

优先做一个简单、可测、可观测的 loop，再根据复杂度引入图编排或 durable runtime。

11. 高频系统设计题

题 1: 设计一个代码修复 Agent。

高分回答结构：

- **输入**：issue、repo、测试命令、约束。
- **环境**：Docker sandbox，每个样本 clean checkout。
- **Loop**：理解 issue -> 定位文件 -> 修改 -> 运行测试 -> 根据失败修复 -> 生成 patch。
- **工具**：grep、read file、edit、run tests、git diff。
- **状态**：已读文件、假设、修改点、测试结果、失败日志。
- **终止**：相关测试通过、最大轮数、无法复现。
- **评测**：SWE-bench 风格，用隐藏/公开测试判 patch。
- **安全**：禁止访问网络或 secrets，限制 shell 命令。
- **观测**：保存 patch、测试日志、工具轨迹、成本。

追问：如果测试很慢怎么办？

- 先跑相关测试，再跑全量 smoke。
- 缓存依赖和构建层。
- 失败日志摘要。
- 超时策略。
- 对 flaky test 复跑确认。

题 2: 设计一个浏览器购物 Agent。

要点：

- 浏览器环境要可 reset，账号/购物车/库存状态要固定。
- action space：click/type/select/navigate/extract。
- 观察：截图、DOM、accessibility tree。
- 成功判据：购物车状态、订单信息，而非最终口头回答。
- 安全：下单/付款前 HITL；限制站点域名；防网页注入。
- eval：WebArena 风格任务集，记录轨迹和最终网页状态。

题 3: 设计一个企业知识库 Agent。

要点：

- 先判断是否需要检索，不能凭空回答。
- 权限过滤必须在检索前或检索中完成，不能检索后靠模型遗忘。
- 引用来源、段落级证据。
- 对冲突证据要说明不确定性。
- eval：答案相关性、faithfulness、引用准确、权限泄露率、拒答准确率。
- 线上 bad case 进入回归集。

题 4: 设计一个客服工具 Agent。

要点：

- policy 是核心：退款、补偿、升级规则。
- 工具动作必须可审计。
- 用户模拟器做多轮评测。
- scorer 检查最终回复、工具动作、数据库状态、policy violation。
- 高风险动作如退款需要审批或限额。
- tau-bench/tau3-bench 是很好的类比。

12. 高频编码题与追问

题: 实现工具调用失败后的恢复策略。

候选人应覆盖：

- 可重试错误：timeout、429、5xx，指数退避。
- 不可重试错误：参数非法、权限不足、业务规则禁止。
- 结构化错误返回给模型： `error_type` 、 `message` 、 `retryable` 、 `suggested_fix` 。
- 达到重试上限后换策略或终止。
- 记录 `trace`。

题: 如何检测 agent 重复调用同一工具？

思路：

- 对 `tool_name + normalized_args` 做 hash。
- 在 sliding window 内统计重复次数。
- 区分幂等查询和有副作用工具。
- 对重复调用注入反馈：已尝试、结果摘要、要求换策略。
- 达到阈值终止或请求人工。

题: 如何做结构化输出验证？

要点：

- Pydantic/JSON Schema 校验。
- 字段级约束：枚举、范围、正则、必填。
- 语义校验：总价是否等于明细之和、日期是否合法。
- 修复 loop：把校验错误反馈给模型。
- 不能无限修复，有 `retry` 上限。

13. 面试官追问清单

如果你是面试官，可以用这些追问区分深度：

1. 你的 agent 什么时候停止？谁决定停止？
2. 你如何复现昨天线上失败的一次 run？
3. 工具返回错数据时，agent 怎么发现？

4. 如果模型升级导致成功率提升但成本翻倍，你怎么判断是否上线？
5. 如何防止网页里的恶意文本控制你的 agent？
6. 用户没有权限的数据，应该在 RAG 的哪个阶段过滤？
7. 你的 eval set 会不会被 prompt 过拟合？
8. LLM judge 和人工标注冲突时听谁的？
9. 多 agent 系统里，哪个 agent 对最终错误负责？
10. 如果一次任务运行 30 分钟，中途 worker 崩了怎么办？

14. 候选人回答模板

架构题模板

1. 先定义任务边界和成功判据。
2. 再决定用 workflow 还是 agent loop。
3. 明确 state、tools、environment、policy。
4. 设计 loop 的终止、错误恢复、预算控制。
5. 设计 harness: dataset、reset、runner、scorer、sandbox、trace。
6. 说明安全权限和 HITL。
7. 说明离线 eval、线上监控、bad case 回流。

Debug 题模板

1. 看是否有版本变更：model/prompt/tool/index/policy。
2. 从 trace 定位失败阶段。
3. 对比成功与失败 trajectory。
4. 复现实验并最小化 case。
5. 加入回归集。
6. 修复后跑分层 eval 和灰度。

评测题模板

1. 程序可判优先程序判。
2. 开放式任务用 rubric + LLM judge + 人工校准。
3. 同时看最终结果、过程合规、成本延迟、安全。
4. 固定版本和环境，保证可复现。
5. 用线上 bad case 持续更新 eval。

15. 资料源与继续阅读

这些资料足够覆盖 2026 年 Agent 面试主线：

- Anthropic, **Building effective agents**: workflow vs agent、简单可组合模式、环境反馈与停止条件。 <https://www.anthropic.com/engineering/building-effective-agents>
- OpenAI, **A practical guide to building agents**: single-agent/multi-agent、run loop、tools、guardrails、handoff。 <https://cdn.openai.com/business-guides-and-resources/a-practical-guide-to-building-agents.pdf>
- OpenAI Agents SDK docs: Agent、Runner、Tools、Guardrails、Sessions、Tracing。 <https://openai.github.io/openai-agents-python/>
- LangGraph docs: StateGraph、conditional edges、checkpoint、durable execution、human-in-the-loop。 <https://docs.langchain.com/oss/python/langgraph/overview>
- MCP Specification 2025-06-18: tools、resources、prompts、sampling、transport、authorization。 <https://modelcontextprotocol.io/specification/2025-06-18>
- Inspect AI docs: Task、Sample、Solver、Scorer、Sandbox、agent eval、running evals。 <https://inspect.aisi.org.uk/>
- SWE-bench / SWE-bench Verified: 真实 GitHub issue、Docker evaluation harness、patch correctness。 <https://www.swebench.com/>
- Terminal-Bench: terminal sandbox、task dataset、test script、oracle solution、execution harness。 <https://www.tbench.ai/>
- OSWorld / OSWorld-Verified: 真实桌面环境、多应用 GUI task、执行式评估。 <https://os-world.github.io/>
- WebArena / VisualWebArena: 自托管网页环境、浏览器 agent benchmark。 <https://webarena.dev/>
- tau-bench / tau3-bench: 客服多轮交互、user simulator、policy、tools、action correctness、voice/knowledge 扩展。 <https://github.com/sierra-research/tau2-bench>

16. 最后一句面试建议

不要把 Agent 讲成“让大模型自己想办法”。更成熟的表达是：

我用受控 loop 让模型在允许的工具和环境中探索；用 harness 复现和判分；用 trace 解释失败；用 guardrails 和权限控制风险；用离线 eval 和线上 bad case 回流持续改进。

这句话背后如果都能展开，基本就是一个合格的 Agent 工程候选人。